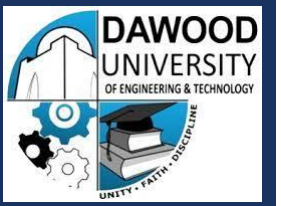
LECTURE – POLYMORPHISM
OOP (JAVA)



MR. MUHAMMAD SALEEM
LECTURER
DEPARTMENT OF COMPUTER SCIENCE

Dawood University of Engineering and Technology, Karachi

AGENDA



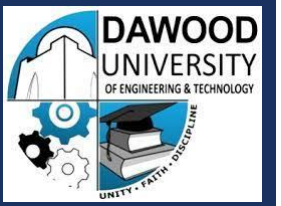
- INTRODUCTION TO POLYMORPHISM
- TYPES OF POLYMORPHISM
- METHOD OVERLOADING
- METHOD OVERRIDING
  - EXAMPLES
- RUN TIME POLYMORPHISM OR DYNAMIC DISPATCH METHOD

POLYMORPHISM



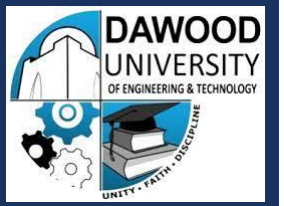
- **Polymorphism in Java** is a concept by which we can perform a *single action in different ways*. Polymorphism is derived from 2 Greek words: poly and morphs.
- The word "poly" means many and "morphs" means forms.
- So, polymorphism means many forms.
- There are two types of polymorphism in Java: compile-time polymorphism and runtime polymorphism. We can perform polymorphism in java by method overloading and method overriding.

TYPES OF POLYMORPHISM



- In Java polymorphism is mainly divided into two types:
  - Compile-time Polymorphism (static)
  - Runtime Polymorphism(dynamic)

METHOD OVERLOADING IN JAVA



- If a class has multiple methods having same name but different in parameters, it is known as **Method Overloading**.
- If we have to perform only one operation, having same name of the methods increases the readability of the program.
- There are two ways to overload the method in java
  1. By changing number of arguments
  2. By changing the data type

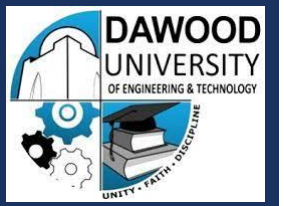
Q) WHY METHOD OVERLOADING IS NOT POSSIBLE BY CHANGING THE RETURN TYPE OF METHOD ONLY?



- In java, method overloading is not possible by changing the return type of the method only because of ambiguity. Let's see how ambiguity may occur

```
class Adder{
static int add(int a,int b){return a+b;}
static double add(int a,int b){return a+b;}
}
class TestOverloading3{
public static void main(String[] args){
System.out.println(Adder.add(11,11));//ambig
uity
}}
```

CAN WE OVERLOAD JAVA MAIN() METHOD?

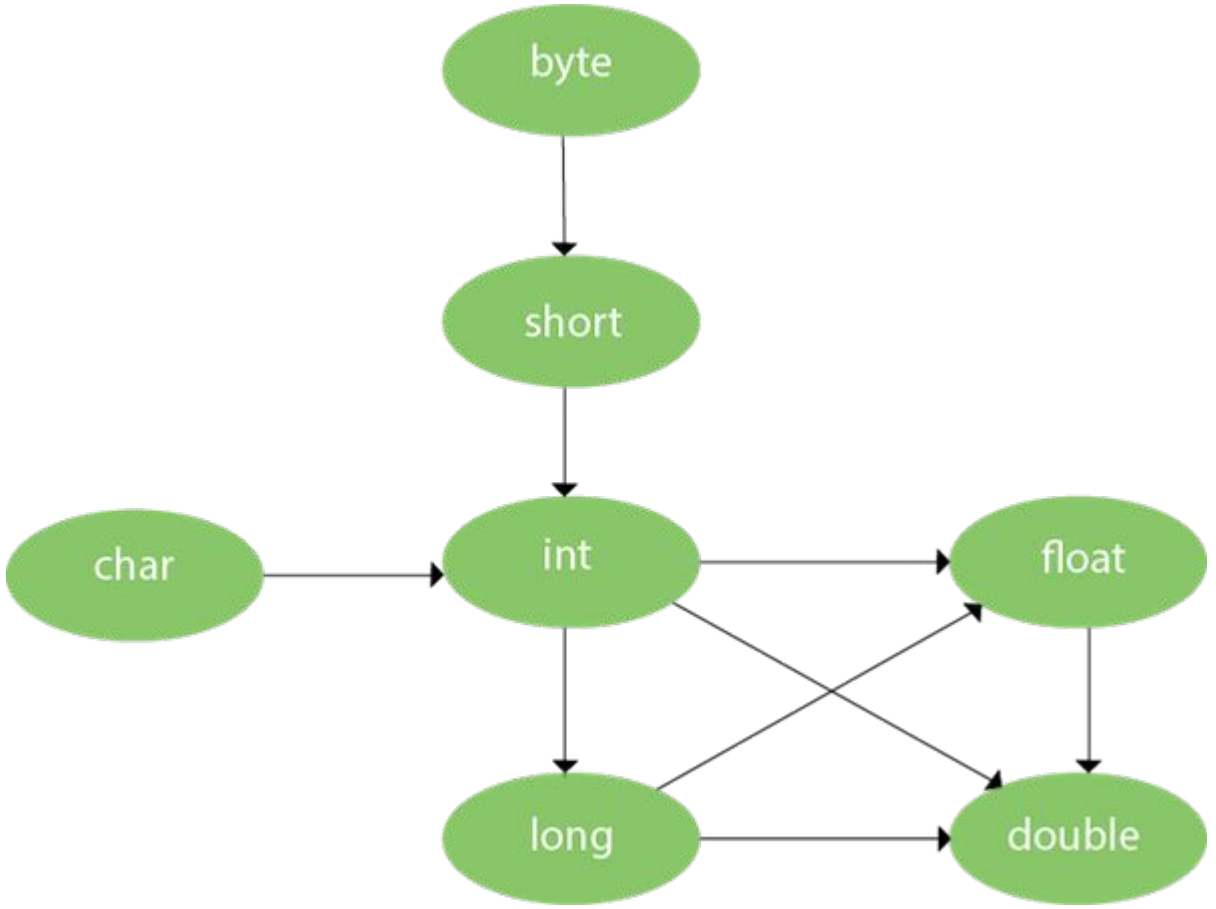


- Yes, by method overloading. You can have any number of main methods in a class by method overloading. But JVM calls main() method which receives string array as arguments only. Let's see the simple example:

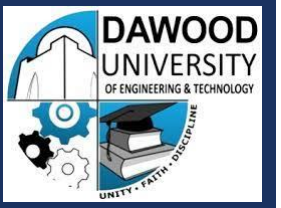
```
class TestOverloading4{
public static void main(String[] args){System.out.println("main with String[]");}
public static void main(String args){System.out.println("main with String");}
public static void main(){System.out.println("main without args");}
}
```

METHOD OVERLOADING AND TYPE PROMOTION

- One type is promoted to another implicitly if no matching datatype is found.
- Let's understand the concept by the figure given below:



METHOD OVERRIDING IN JAVA



- If subclass (child class) has the same method as declared in the parent class, it is known as **method overriding in Java**.
- In other words, If a subclass provides the specific implementation of the method that has been declared by one of its parent class, it is known as method overriding.
- Method overriding is used to provide the specific implementation of a method which is already provided by its superclass.
- Method overriding is used for runtime polymorphism

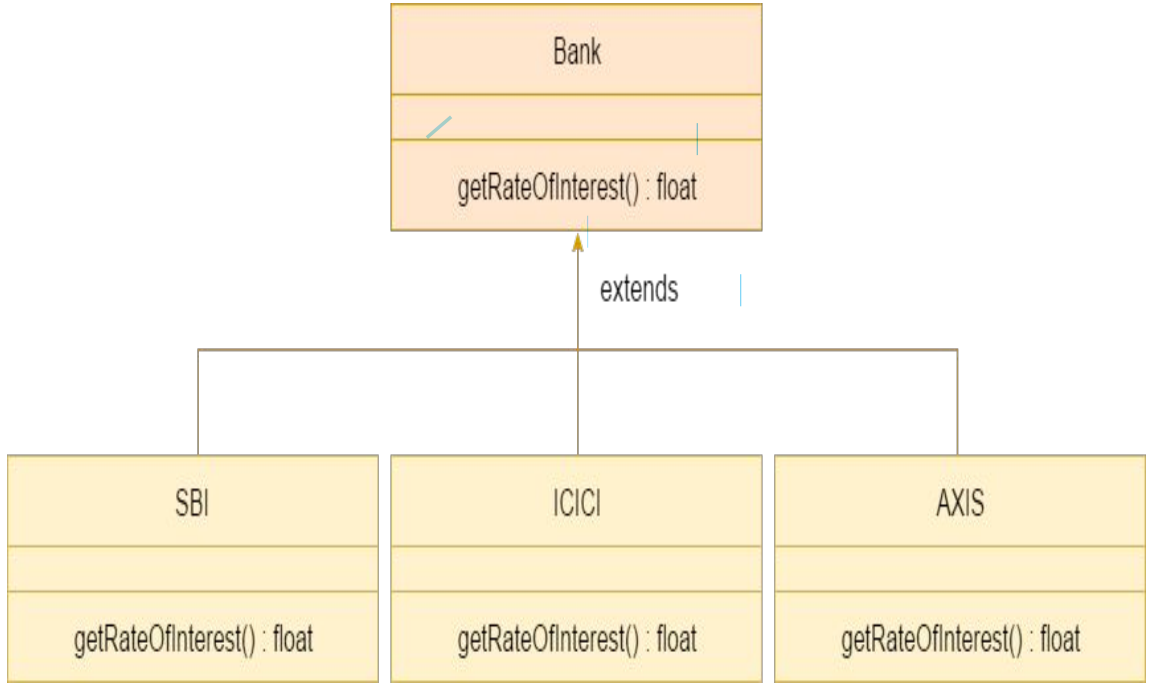
RULES FOR JAVA METHOD OVERRIDING



1. The method must have the same name as in the parent class
2. The method must have the same parameter as in the parent class.
3. There must be an IS-A relationship (inheritance).

A REAL EXAMPLE OF JAVA METHOD OVERRIDING

- Consider a scenario where Bank is a class that provides functionality to get the rate of interest.
- However, the rate of interest varies according to banks. For example, SBI, ICICI and AXIS banks could provide 8%, 7%, and 9% rate of interest.

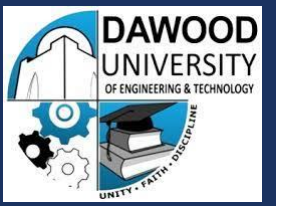


SOME IMPORTANT QUESTIONS?



- Can we override static method?
  - No, a static method cannot be overridden. It can be proved by runtime polymorphism.
  - Why can we not override static method?
  - It is because the static method is bound with class whereas instance method is bound with an object. Static belongs to the class area, and an instance belongs to the heap area.
- Can we override java main method?
  - No, because the main is a static method.

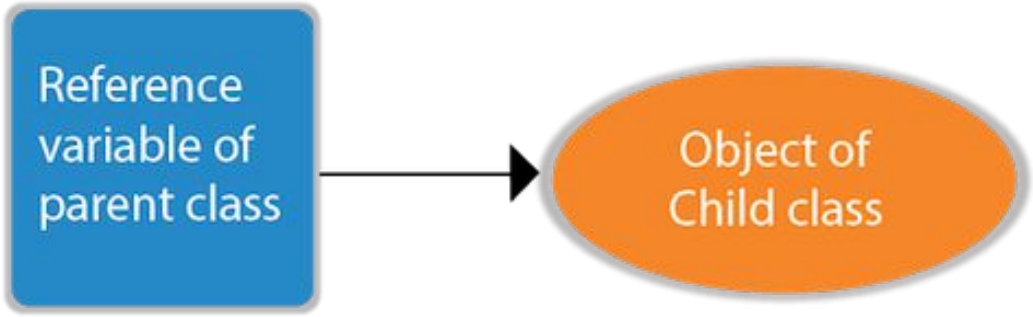
RUNTIME POLYMORPHISM IN JAVA



- **Runtime polymorphism** or **Dynamic Method Dispatch** is a process in which a call to an overridden method is resolved at runtime rather than compile-time.
- In this process, an overridden method is called through the reference variable of a superclass.
- The determination of the method to be called is based on the object being referred to by the reference variable.
- Let's first understand the upcasting before Runtime Polymorphism.

UPCASTING

- If the reference variable of Parent class refers to the object of Child class, it is known as upcasting.



EXAMPLE OF RUN TIME POLYMORPHISM



- In this example, we are creating two classes Bike and Splendor.
- Splendor class extends Bike class and overrides its run() method.
- We are calling the run method by the reference variable of Parent class.
- Since it refers to the subclass object and subclass method overrides the Parent class method, the subclass method is invoked at runtime.
- Since method invocation is determined by the JVM not compiler, it is known as runtime polymorphism.

```
class Bike{
  void run(){System.out.println("running");}
}
class Splendor extends Bike{
  void run(){System.out.println("running safely with 60 km");}

  public static void main(String args[]){
    Bike b = new Splendor();//upcasting
    b.run();
  }
}
```




```
class Shape{
void draw(){System.out.println("drawing...");}
}
class Rectangle extends Shape{
void draw(){System.out.println("drawing rectangle...");}
}
class Circle extends Shape{
void draw(){System.out.println("drawing circle...");}
}
class Triangle extends Shape{
void draw(){System.out.println("drawing triangle...");}
}
class TestPolymorphism2{
public static void main(String args[]){
Shape s;
s=new Rectangle();
s.draw();
s=new Circle();
s.draw();
s=new Triangle();
s.draw();  }  }
```



THANK YOU